UNIT 1 TOPICS:

Java Fundamentals – Introduction to Java-Overview of Java, its features (platform-independent, secure, robust), architecture, Compilation & Execution process, **Data types**-Primitive types (int, float, char, etc.), reference types, and type conversion, **Control Structures** - if-else, switch, for, while, do-while – controlling the program flow (break, continue, return), **Classes and Objects** - Blueprint of objects, Object creation using ***new***, Syntax and structure of a class encapsulation of attributes and methods, instantiation, Access modifiers (public, private, default).

1. Java Fundamentals

1.1 Introduction to Java

Java is a **high-level, object-oriented, class-based programming language** developed by Sun Microsystems (now Oracle). It follows the principle **"Write Once, Run Anywhere (WORA)"**, meaning Java programs can run on any system with a Java Virtual Machine (JVM).

Java is widely used for:

- Desktop applications
- Web applications
- Mobile applications (Android)
- Enterprise systems
- Embedded and IoT systems

Why Java is called a High-Level, Object-Oriented, Class-Based Programming Language

1 Java is a High-Level Programming Language

Definition (simple):
A high-level language is easy for humans to read, write, and understand. It hides hardware details like memory and registers.

Why Java fits:

- English-like syntax
- No manual memory handling (no pointers like C)
- Platform independent (Write Once, Run Anywhere)

Example:

```
int a = 10;
int b = 20;
int sum = a + b;
System.out.println(sum);
```

👉 You don't worry about **how memory is allocated** or **CPU instructions**. Java handles it.

1️⃣ Java as a High-Level Language

```
scss

Programmer (Human)
        ↓
     Java Code
        ↓
  Java Virtual Machine (JVM)
        ↓
     Computer Hardware
```

👉 *"Java acts as a bridge between humans and machines. We don't talk to hardware directly."*

2️⃣ Java is an Object-Oriented Programming (OOP) Language

Definition:
Object-Oriented Programming organizes software using **objects** that contain **data (variables)** and **behavior (methods)**.

Java supports all major OOP principles:

- Encapsulation
- Inheritance
- Polymorphism
- Abstraction

Real-life analogy:
A **Car** is an object

- Data → color, speed, model
- Behavior → start(), stop(), accelerate()

Example:

```
class Car {
    String color;
    int speed;

    void drive() {
        System.out.println("Car is driving");
    }
}
```

2 Java as an Object-Oriented Language

```
pgsql

Real World        Java Program
----------        ------------
Car         →     Car Object
Student     →     Student Object
Bank        →     BankAccount Object
```

Concept:

- Object = Data + Methods
- Java programs are built around **objects**, not functions.

3 Java is a Class-Based Language

Definition:
In Java, **everything is created from a class**. A class is a blueprint; objects are real instances.

Key rule:
➡️ **No object without a class**

Example:

```java
class Student {
    String name;
    int rollNo;

    void study() {
        System.out.println("Student is studying");
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student();
        s1.name = "Arun";
        s1.rollNo = 101;
        s1.study();
    }
}
```

```
Class (Blueprint)
-----------------
Student
- name
- rollNo
+ study()

        ↓

Object (Real)
-------------
s1
name = "Arun"
rollNo = 101
```

👉 *"No class → No object → No program in Java."*

4️⃣ Putting It All Together (One-Line Explanation)

Java is a high-level language because it is easy to read and machine-independent, **object-oriented** because it uses objects to model real-world entities, and **class-based** because all objects are created from classes.

Software Required to Run a Java Program

1️⃣ JDK – Java Development Kit (Mandatory)

- Contains:
  - **javac** (compiler)
  - **JRE**
  - Java libraries
- Needed to **write + compile + run** Java programs

2️⃣ JRE – Java Runtime Environment

- Contains:
  - **JVM**
  - Core libraries
- Needed to **run** Java programs

3️⃣ IDE / Text Editor (Optional but Recommended)

- VS Code
- Eclipse
- IntelliJ IDEA
- Notepad (basic)

Java Installation – Diagram Explanation

```
Download JDK
     ↓
Install JDK
     ↓
Set PATH
     ↓
java -version
```

👉 *"Without JDK, Java programs cannot be compiled or executed."*

Compilation Process (Diagram-Based)

```
Hello.java   (Source Code)
     ↓
javac Hello.java   (Compiler)
     ↓
Hello.class  (Bytecode)
```

- Compiler checks syntax
- Converts code into **platform-independent bytecode**

Execution Process (Diagram-Based)

```
Hello.class (Bytecode)
     ↓
JVM
     ↓
Machine Code
     ↓
Output
```

Key point:
👉 JVM converts bytecode into machine code **based on OS**.

Complete Java Program Flow (Very Important Diagram)

```
Editor
  ↓
.java file
  ↓
javac (Compiler)
  ↓
.class (Bytecode)
  ↓
JVM
  ↓
Output
```

Simple Example for Students

```java
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello Java");
    }
}
```

Commands:

```
javac Hello.java
java Hello
```

Summary:

Java program execution involves installing JDK, compiling source code into bytecode using javac, and executing it using JVM to produce output.

Important clarification first

👉 **Java does NOT have header files** like C or C++.

But Java uses **`import` statements** instead of header files.

So when students say *"header file in Java"*, what they actually mean is:

- `import` statements (for built-in libraries)

- OR basic program structure

Simple Java Program (with “header-like” import)

```
// Import statement (acts like header file)
import java.util.Scanner;

public class SimpleProgram {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter your name: ");
        String name = sc.nextLine();

        System.out.println("Hello " + name);
    }
}
```

Explain to Students (Very Simple)

1️⃣ import statement (Header file equivalent)

```
import java.util.Scanner;
```

- Used to include **predefined classes**
- Similar purpose to `#include<stdio.h>` in C
- But Java **does not copy code**, it just makes classes available

---

2️⃣ class declaration

```
public class SimpleProgram
```

- Every Java program **must have a class**
- File name must be **SimpleProgram.java**

3️⃣ main method

```
public static void main(String[] args)
```

- Program execution starts here

- JVM looks for this method first

4️⃣ Output statement

```
System.out.println("Hello " + name);
```

- Prints output to screen

Diagram-Based Comparison (C vs Java)

```
C Program                       Java Program
---------                       ------------
#include<stdio.h>      →       import java.util.*;
main()                 →       class + main()
```

Very Important Exam Line (Use This)

Java does not use header files. Instead, it uses import statements to access predefined classes and packages.

Ultra-Simple Java Program (No import)

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello Java");
    }
}
```

(No header / import needed here)

1.2 Overview of Java

Key characteristics:

- Simple and easy to learn

- Object-oriented
- Platform independent
- Secure and robust
- Multithreaded and portable

1.3 Features of Java

a) Platform Independent

- Java code is compiled into **bytecode** (.class file)
- Bytecode runs on **JVM**, not directly on hardware
- Same program runs on Windows, Linux, Mac without modification

b) Secure

- No explicit use of pointers
- Bytecode verification before execution
- Security manager controls access to system resources

c) Robust

- Strong memory management
- Automatic garbage collection
- Exception handling mechanism
- Type checking at compile time

1.4 Java Architecture

Main Components:

1. **Java Source Code (.java)**
2. **Java Compiler (javac)** – converts source code to bytecode
3. **Bytecode (.class)** – intermediate code
4. **Java Virtual Machine (JVM)** – executes bytecode
5. **JRE (Java Runtime Environment)** – JVM + libraries
6. **JDK (Java Development Kit)** – JRE + development tools

1.5 Compilation and Execution Process

Steps:

1. Write program in a `.java` file
2. Compile using `javac ProgramName.java`
3. Bytecode file `ProgramName.class` is generated
4. Run using `java ProgramName`
5. JVM interprets and executes bytecode

Example:

```
class Hello {
public static void main(String[] args) {
System.out.println("Hello Java");
}
}
```

2. Data Types in Java

Data types specify the **type of data** a variable can hold.

2.1 Primitive Data Types

Data Type	Size	Example
byte	1 byte	byte b = 10;
short	2 bytes	short s = 100;
int	4 bytes	int x = 50;
long	8 bytes	long l = 1000L;
float	4 bytes	float f = 10.5f;
double	8 bytes	double d = 20.75;
char	2 bytes	char c = 'A';
boolean	1 bit	boolean flag = true;

2.2 Reference Data Types

- Store **addresses of objects**
- Created using `new` keyword

Examples:

- Class objects
- Arrays
- Strings
- Interfaces

```java
String name = "Java";
int[] marks = new int[5];
```

2.3 Type Conversion

a) Implicit Type Casting (Widening)

- Automatically done by Java
- Smaller type → larger type

```java
int a = 10;
double b = a;
```

b) Explicit Type Casting (Narrowing)

- Done manually

```java
double x = 10.5;
int y = (int)x;
```

3. Control Structures

Control structures control the **flow of execution** in a program.

3.1 Conditional Statements

a) if-else

```
if (marks >= 50) {
    System.out.println("Pass");
} else {
    System.out.println("Fail");
}
```

b) switch

```
switch(day) {
    case 1: System.out.println("Monday"); break;
    case 2: System.out.println("Tuesday"); break;
    default: System.out.println("Invalid");
}
```

3.2 Looping Statements

a) for loop

```
for(int i = 1; i <= 5; i++) {
    System.out.println(i);
}
```

b) while loop

```
int i = 1;
while(i <= 5) {
    System.out.println(i);
    i++;
}
```

c) do-while loop

```java
int i = 1;
do {
    System.out.println(i);
    i++;
} while(i <= 5);
```

3.3 Branching Statements

a) break

- Terminates loop or switch

b) continue

- Skips current iteration

c) return

- Returns control to calling method

```java
if(x < 0) {
    return;
}
```

4. Classes and Objects

4.1 Class

A **class** is a blueprint for creating objects. It defines:

- Data members (variables)
- Member functions (methods)

```java
class Student {
    int rollNo;
    String name;

    void display() {
        System.out.println(rollNo + " " + name);
    }
}
```

4.2 Object

An **object** is an instance of a class.

Object creation using `new`:

```
Student s1 = new Student();
s1.rollNo = 1;
s1.name = "Arun";
s1.display();
```

4.3 Encapsulation

- Binding data and methods together
- Achieved using **classes** and **access modifiers**

4.4 Access Modifiers

Modifier	Accessibility
public	Accessible everywhere
private	Accessible only within class
default	Accessible within same package

Example:

```
class Account {
    private int balance;

    public void setBalance(int b) {
        balance = b;
    }

    public int getBalance() {
        return balance;
    }
}
```

5. Summary

- Java is platform independent and object-oriented
- JVM plays a key role in execution
- Data types define data storage
- Control structures manage program flow
- Classes and objects are the core of OOP
- Encapsulation improves security and maintainability